MIT 8103 Advanced Database Systems: Portfolio Summary

Case Study: Fashion Accessories Ecommerce System
Name: Funmilayo Olalekan
Student ID: 301928246
Repository: https://github.com/fummyolalekan/Olalekan_Funmilayo_301928246

Case study

The case study used throughout the portfolios was a Fashion Accessories Ecommerce System. The system manages customers, products, orders, payments, and inventory for an online retailer of fashion accessories. Customers can register, place orders, make payments, and track purchases, while administrators manage stock levels, product information, order fulfilment, and business reporting.

Portfolio 1: Database Design and Modelling

Portfolio 1 focused on relational database design using PostgreSQL. I designed six entities: customer, category, product, order, order_item, and payment. The design included primary keys, foreign keys, unique constraints, and check constraints based on the business rules.

An ER diagram was created to show relationships between entities, and the design was normalised to Third Normal Form (3NF). This reduced duplicate data and ensured that information was stored in the most appropriate table. The schema was implemented successfully and tested using sample data. Constraint tests confirmed that the database correctly rejected duplicate customer emails, duplicate category names, invalid foreign keys, missing required values, negative stock values, and duplicate payments. These results showed that the design enforced data integrity and supported the required business rules.

Portfolio 2: Query Processing and Optimisation

Portfolio 2 focused on SQL queries and database performance. Three business queries were developed and tested:

- A join query showing customers, orders, and ordered products.
- A query using NOT EXISTS to identify orders without payments.
- A grouped query using LEFT JOIN to count products by category.

Each query returned the expected results from the sample dataset and demonstrated the use of joins, filtering, aggregation, and grouping.

For performance testing, a benchmark table containing 100,000 synthetic records was created. A date-range query was analysed before and after adding a B-tree index on order_date.

Measurement	**Before Index**	**After Index**
Scan Type	Sequential Scan	Bitmap Index Scan + Bitmap Heap Scan
Execution Times (ms)	7.635, 8.314, 8.414	2.255, 2.507, 2.416
Median Time (ms)	8.314	2.416
Shared Buffer Hits	935	181

The results showed that the index reduced execution time and buffer usage. However, the benchmark used generated test data and a single query pattern, so the measurements do not guarantee the same improvement for every workload.

Portfolio 3: Transactions and Concurrency

Portfolio 3 focused on transaction control and concurrency.

The first exercise demonstrated the difference between COMMIT and ROLLBACK. A valid transaction updated an order from Pending to Shipped and was committed successfully. A second transaction deliberately violated a check constraint by attempting to set stock quantity to a negative value. PostgreSQL rejected the operation and the transaction was rolled back. Verification queries confirmed that the committed change remained while the invalid stock update was removed.

A second exercise examined concurrent access by using two PostgreSQL sessions running with READ COMMITTED isolation. Session A inserted a payment without committing immediately. Session B could not see the uncommitted payment and attempted to insert a conflicting payment for the same order. PostgreSQL delayed Session B until Session A committed. Session B then received a duplicate-key error because the database enforced the unique constraint on payment.order_id.

These exercises showed how transaction management, isolation levels, and constraints work together to maintain consistency in a multi-user environment.

Portfolio 4: NoSQL and Advanced Data Models

Portfolio 4 introduced MongoDB and document databases. A local MongoDB database called fashion_accessories_portfolio was created with a single collection named activity_events. Four example documents were inserted using embedded documents to store customer and event information.

Two event types were represented: ORDER_CREATED & PAYMENT_RECORDED

Several MongoDB queries were tested successfully. The collection contained four documents, a customer history query returned two events for customer ID 1, and a nested-field query returned the event where the order status was Shipped.

MongoDB was suitable for this scenario because different event types could share common fields while storing different detail structures within the same collection. This provided flexibility without requiring relational joins. A limitation is that MongoDB does not automatically enforce foreign-key style relationships in the same way as PostgreSQL.

Portfolio 5: Distributed and Cloud Database Exercise

Portfolio 5 was a discussion-based exercise exploring distributed database concepts using MongoDB.

The actual implementation remained a local MongoDB deployment and did not include replication or sharding. Instead, a proposed production design was discussed.

The proposal used a three-member MongoDB replica set distributed across separate locations. The design examined primary and secondary server roles, automatic elections, majority write acknowledgement, and data replication.

A failure scenario was considered in which the primary server becomes unavailable. If a majority of voting members remains operational, MongoDB can elect a new primary and continue processing writes. If a majority is unavailable, write operations cannot continue until quorum is restored.

The discussion also distinguished replication from sharding. Replication copies data to improve availability, whereas sharding distributes data across servers to support larger workloads. Based on the small portfolio dataset, sharding was unnecessary, although it could become useful in a larger ecommerce environment. Cloud considerations such as authentication, encryption, backups, access control, and operational costs were also discussed.

The distributed architecture described in Portfolio 5 was proposed only and was not implemented or tested.

Evidence

All portfolio deliverables, SQL scripts, MongoDB scripts, query results, optimisation tests, transaction exercises, and screenshots are available in the project repository:

Repository: https://github.com/fummyolalekan/Olalekan_Funmilayo_301928246

The repository contains:

- Database design documentation
- ER diagram and normalisation analysis
- PostgreSQL schema and sample data
- Query processing and optimisation files
- Transaction and concurrency scripts
- MongoDB implementation files
- Supporting screenshots and test results

Individual reflection

One important design choice was creating a separate order_item table to support the many-to-many relationship between orders and products. This allowed one order to contain multiple products while also allowing products to appear in multiple orders. I also used a unique constraint on payment.order_id to ensure that only one payment could be recorded for each order.

The most challenging parts of the portfolios were transaction isolation, concurrency control, and debugging SQL constraint errors. Understanding how PostgreSQL manages locks and transaction visibility required multiple attempts before the behaviour became clear.

To overcome these challenges, I practised the exercises several times, repeatedly tested SQL commands, consulted PostgreSQL documentation, and used screenshots to verify the outcome of each stage. I also re-ran the concurrency exercises in separate sessions until I understood the locking and waiting behaviour correctly.

The main lesson I learned was how database design, constraints, transactions, and indexing work together to maintain data integrity and performance. I also learned how concurrency controls prevent conflicting updates in multi-user database systems.

References

- PostgreSQL Documentation: https://www.postgresql.org/docs/current/
- MongoDB Documentation: https://www.mongodb.com/docs/
- MongoDB Shell Documentation: https://www.mongodb.com/docs/mongodb-shell/
- MongoDB Replication Documentation: https://www.mongodb.com/docs/manual/replication/
- MongoDB Write Concern Documentation: https://www.mongodb.com/docs/v8.0/reference/write-concern/
- MongoDB Sharding Documentation: https://www.mongodb.com/docs/manual/sharding/

AI-use declaration

Tools used: Microsoft Copilot (GPT-5), PostgreSQL documentation, and MongoDB documentation.

Help received: Microsoft Copilot assisted with reviewing database design decisions, explaining normalisation concepts, generating draft SQL examples, discussing transaction and concurrency scenarios, preparing MongoDB document examples, and improving portfolio structure and wording.

My verification: I personally reviewed all generated content, implemented the PostgreSQL schema, executed SQL scripts, tested constraints, measured query performance, completed transaction and concurrency exercises, executed MongoDB commands, verified outputs, captured screenshots, corrected errors encountered during implementation, and confirmed that the submitted results matched the actual database behaviour.